CO2_AT3

Question 1: Online Food Delivery Data Analysis

Aim: To analyze food delivery orders and improve delivery performance and customer satisfaction.

A. Using Pandas to read, clean, and filter data

Pandas is used to read order data from a CSV file, display records, and select orders based on delivery time or customer ratings.

B. Data preprocessing

- Fill missing delivery times using the median.
- Remove duplicate orders.
- Remove incorrect prices that are zero or negative.
- Filter orders with long delivery times.

C. Visualization

Matplotlib and Seaborn are used to display delivery delays, top restaurants, and customer ratings.

Python program

import pandas as pdimport matplotlib.pyplot as pltimport seaborn as snsdata = { "Order_ID": [1, 2, 3, 4, 4, 6], "Restaurant": ["A", "B", "A", "C", "C", "B"], "Delivery_Time": [30, 45, None, 60, 60, 35], "Price": [200, 300, 150, 250, 250, -50], "Rating": [5, 4, 4, 3, 3, 5]}df = pd.DataFrame(data)# Remove duplicate ordersdf = df.drop_duplicates(subset="Order_ID")# Remove incorrect pricesdf = df[df["Price"] > 0].copy()# Fill missing delivery timedf["Delivery_Time"] = df["Delivery_Time"].fillna(df["Delivery_Time"].median())# Display cleaned dataprint("Cleaned Data:")print(df)# Filter delayed ordersprint("\nDelayed Orders:")print(df[df["Delivery_Time"] > 40])# Top restaurants by number of ordersdf["Restaurant"].value_counts().plot(kind="bar")

Observations

1. Pandas reads and filters order records easily.
2. Duplicate orders and incorrect prices are removed.
3. Missing delivery times are filled using the median.
4. Delivery-time charts help identify delays.
5. Restaurant order counts and customer ratings help measure performance.

Conclusion: Data preprocessing and visualization help the company identify delivery delays, popular restaurants, and customer satisfaction levels.

Question 2: Bank Loan Approval Analysis

Aim: To analyze loan applications and identify factors related to loan approval.

A. Using Pandas DataFrame operations

Pandas selects important columns such as income, loan amount, credit score, and approval status. It can also filter approved and rejected applications.

B. Data preprocessing

- Fill missing income values using the median.
- Convert gender and employment type into numerical values using one-hot encoding.
- Normalize loan amounts to a range between 0 and 1 using Min-Max scaling.

C. Visualization

Scatter plots and box plots compare income, credit scores, and loan amounts for approved and rejected applications.

Python program

import pandas as pdimport matplotlib.pyplot as pltimport seaborn as snsfrom sklearn.preprocessing import MinMaxScalerdata = { "Income": [40000, 25000, None, 60000, 30000, 50000], "Loan_Amount": [200000, 150000, 300000, 250000, 180000, 220000], "Credit_Score": [720, 580, 750, 710, 600, 690], "Gender": ["Male", "Female", "Male", "Female", "Male", "Female"], "Employment": ["Salaried", "Self", "Salaried", "Salaried", "Self", "Salaried"], "Status": ["Approved", "Rejected", "Approved", "Approved", "Rejected", "Approved"]}df = pd.DataFrame(data)# Select required columnsprint(df[["Income", "Loan_Amount", "Credit_Score", "Status"]])# Fill missing incomedf["Income"] = df["Income"].fillna(df["Income"].median())# Encode categoriesdf = pd.get_dummies(df, columns=["Gender", "Employment"], dtype=int)# Normalize loan amountscaler = MinMaxScaler()df["Loan_Amount_Normalized"] = scaler.fit_transform(df[["Loan_Amount"]])

Observations

1. Pandas selects the required loan application columns.
2. Missing income values are filled using the median.
3. Gender and employment categories are converted into numerical columns.
4. Loan amounts are normalized between 0 and 1.
5. The charts help compare the financial details of approved and rejected applicants.

Conclusion: Income, credit score, and loan amount can help the bank understand loan approval patterns. These factors alone do not determine whether a loan should be approved.

Question 3: Smart Energy Consumption Monitoring

Aim: To analyze electricity usage by appliance and identify ways to save energy.

A. Using Pandas to group data

Pandas groups readings by appliance and date. The groupby() function calculates total energy consumption for each appliance.

B. Data preprocessing

- Fill missing readings using the median.
- Remove negative or abnormal readings.
- Calculate total energy consumption.

C. Visualization

Line charts show daily energy usage, while bar charts compare appliance consumption.

Python program

import pandas as pdimport matplotlib.pyplot as pltimport seaborn as snsdata = { "Date": ["2026-10-01", "2026-10-01", "2026-10-01", "2026-10-02", "2026-10-02", "2026-10-02", "2026-10-03", "2026-10-03", "2026-10-03"], "Appliance": ["AC", "Fan", "Light", "AC", "Fan", "Light", "AC", "Fan", "Light"], "Energy": [5, 2, 1, 6, None, 1, 7, 2, -3]}df = pd.DataFrame(data)df["Date"] = pd.to_datetime(df["Date"])# Remove negative readingsdf.loc[df["Energy"] < 0, "Energy"] = None# Fill missing readingsdf["Energy"] = df["Energy"].fillna(df.groupby("Appliance")["Energy"].transform("median"))# Group by applianceappliance_total = df.groupby("Appliance")["Energy"].sum()print("Energy by

Appliance:")print(appliance_total)# Group by datedaily_total = df.groupby("Date")["Energy"].sum()print("\nDaily Energy Usage:")print(daily_total)

Observations

1. Pandas groups electricity readings by appliance and date.
2. Negative energy readings are treated as invalid.
3. Missing readings are filled using the appliance median.
4. The bar chart identifies appliances with high energy consumption.
5. The line chart shows changes in daily electricity usage.

Conclusion: Grouping and visualization help identify high-energy appliances and daily usage patterns. The results can help households reduce unnecessary electricity consumption.

Note: In real systems, abnormal readings should be checked against meter limits before being removed.

Question 4: Online Learning Platform Performance Analysis

Aim: To analyze student activity, quiz scores, and course completion.

A. Using Pandas to sort, filter, and rank

Pandas sorts students by quiz score, filters students based on completion percentage, and ranks them according to performance.

B. Data preprocessing

- Fill missing quiz scores using the median.
- Convert login dates into the correct date format.
- Create a performance category based on quiz marks.

C. Visualization

Seaborn and Matplotlib show quiz score distributions, student engagement, and course completion status.

Python program

import pandas as pdimport matplotlib.pyplot as pltimport seaborn as snsdata = { "Student": ["A", "B", "C", "D", "E", "F"], "Login_Date": ["2026-10-01", "2026-10-02", "2026-10-03", "2026-10-04", "2026-10-05", "2026-10-06"], "Quiz_Score": [85, 60, None, 95, 45, 75], "Completion": [90, 65, 80, 100, 40, 85]}df = pd.DataFrame(data)# Convert datedf["Login_Date"] = pd.to_datetime(df["Login_Date"])# Fill missing quiz scoresdf["Quiz_Score"] = df["Quiz_Score"].fillna(df["Quiz_Score"].median())# Create performance categorydf["Category"] = df["Quiz_Score"].apply(lambda x: "High" if x >= 80 else ("Medium" if x >= 60 else "Low"))# Rank studentsdf["Rank"] = df["Quiz_Score"].rank(ascending=False, method="min").astype(int)# Sort students by scoreprint("Ranked Students:")

Observations

1. Pandas sorts students according to quiz scores.
2. Missing quiz scores are filled using the median.
3. Login dates are converted into date format.
4. Performance categories help identify high-, medium-, and low-performing students.
5. Charts compare quiz scores and course completion percentages.

Conclusion: Student performance analysis helps identify students who may need additional support and helps the platform improve course engagement.

Question 5: Public Transport Passenger Data Analysis

Aim: To analyze passenger counts and identify busy routes and peak travel hours.

A. Using Pandas to analyze passenger data

Pandas groups passenger counts by route, bus stop, and time slot. This helps identify the busiest routes and stops.

B. Data preprocessing

- Remove invalid route numbers.
- Fill missing passenger counts using the median.
- Remove duplicate records.

C. Visualization

Bar charts compare passenger counts across routes, while line charts show changes in passenger numbers over time.

Python program

import pandas as pdimport matplotlib.pyplot as pltimport seaborn as snsdata = { "Route": ["R1", "R2", "R1", "R3", "R2", "R1", "R3", "R2"], "Stop": ["A", "B", "C", "A", "C", "B", "B", "A"], "Time": ["08:00", "08:00", "09:00", "09:00", "10:00", "10:00", "11:00", "11:00"], "Passengers": [50, 70, 60, 45, None, 80, 40, 65]}df = pd.DataFrame(data)# Remove invalid route numbersdf = df[df["Route"].str.match(r"R[1-9]\d*", na=False)]# Remove duplicate recordsdf = df.drop_duplicates()# Fill missing passenger countsdf["Passengers"] = df["Passengers"].fillna(df["Passengers"].median())print("Cleaned Data:")print(df)# Passenger count by routeroute_total = df.groupby("Route")["Passengers"].sum()print("\nPassengers by Route:")print(route_total)# Passenger count by stop

Observations

1. Pandas groups passenger counts by route, stop, and time.
2. Invalid route numbers are removed.
3. Missing passenger counts are filled using the median.
4. The route chart identifies routes with higher passenger totals.
5. The time chart helps identify periods with higher passenger demand.

Conclusion: Passenger data analysis helps transport departments identify busy routes and peak travel periods. This information can support better bus scheduling and reduce overcrowding.